

SIMATS ENGINEERING

Department of Computer Science and Engineering
ASSESSMENT TOOL 2 - CONCEPT MAPPING

Course Code:	CSA1205	Course Title:	Computer Architecture for Multicore Systems
CO Assessed:	CO1	Assessment:	Assessment Tool 2 - Concept Mapping
Weightage:	35%	Total Marks:	25
Student Name:	V JEEVANANTHAM	Reg. No.:	192421234

CO1 Statement: Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4).

Code of Conduct

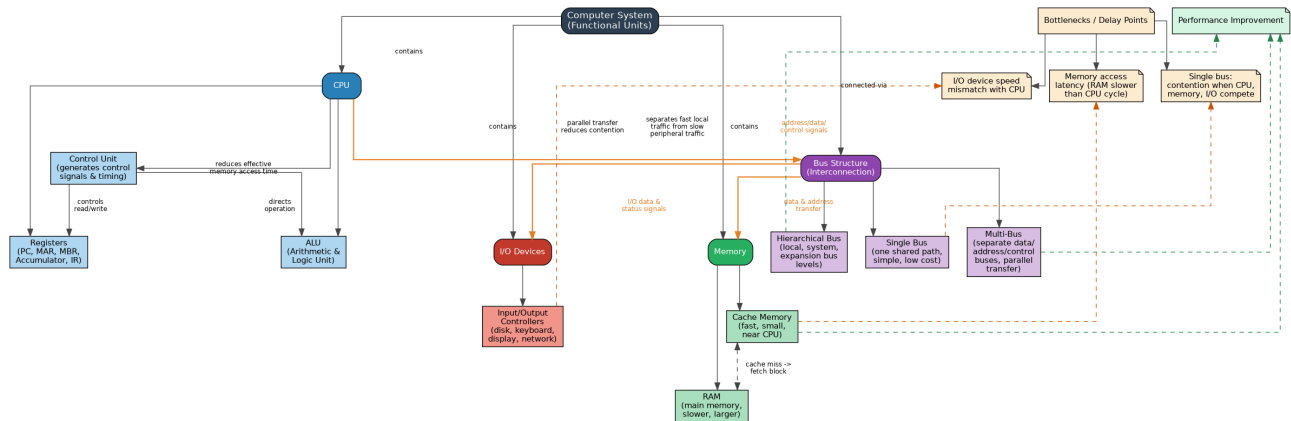
I, **V JEEVANANTHAM** (Reg. No.: **192421234**), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Questions Answered: Q1 and Q2 (as per Annexure A)

Q1. CPU, Memory, I/O and Bus Structures - Concept Map

Construct a detailed concept map showing the relationship between CPU (ALU, Control Unit, Registers), memory (cache, RAM), and I/O devices. Extend the map to include single-bus, multi-bus, and hierarchical bus structures, and clearly illustrate how data and control signals flow among these components during execution. Indicate possible points of delay or bottlenecks and how different bus structures help in improving performance.



Explanation of the Concept Map

1. CPU Components

The **ALU** performs all arithmetic and logical operations. The **Control Unit (CU)** generates the timing and control signals that direct the ALU and registers, sequencing every step of instruction execution. **Registers** (PC, MAR, MBR, Accumulator, IR) hold addresses, data, and instructions temporarily for very fast access.

2. Memory Hierarchy

Cache memory sits close to the CPU and stores frequently used data/instructions for very low access time. **RAM** is larger but slower main memory. On a cache miss, a block is fetched from RAM into cache, which is a major source of delay.

3. I/O Devices

I/O controllers (disk, keyboard, display, network) interface peripheral devices with the CPU/memory. Their speed is usually far lower than the CPU, so they use interrupts or DMA to avoid wasting CPU cycles.

4. Bus Structures

A **single bus** is a shared path for address, data, and control signals - simple and low-cost, but only one transfer can happen at a time, causing contention. A **multi-bus** design uses separate buses for address, data, and control, allowing more parallelism. A **hierarchical bus** separates a fast local/system bus (CPU-memory traffic) from a slower expansion bus (I/O traffic), so slow peripherals do not hold up CPU-memory transfers.

5. Data and Control Flow

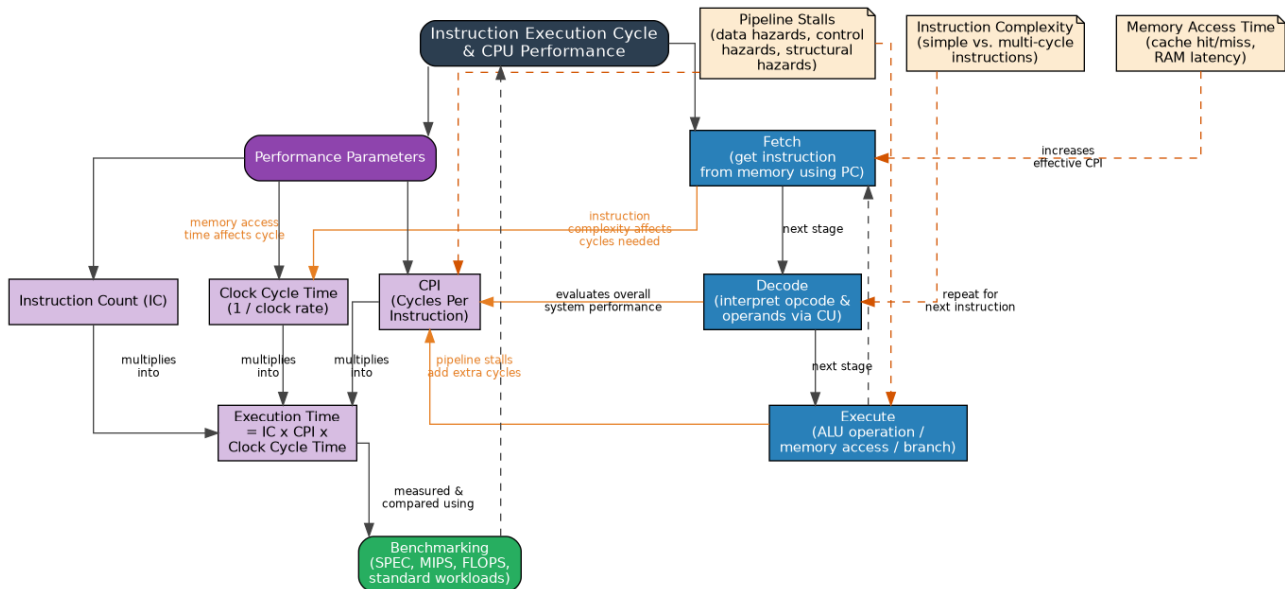
During execution, the CU issues control signals over the control bus, addresses travel on the address bus/lines, and data moves on the data bus between CPU, memory, and I/O. Every fetch, decode, and execute step depends on this signal and data movement.

6. Bottlenecks and Performance

The main bottlenecks are: bus contention on a single shared bus when CPU, memory, and I/O compete for access; memory access latency because RAM is slower than CPU cycle time; and I/O speed mismatch with the CPU. Multi-bus and hierarchical bus structures reduce these bottlenecks by allowing parallel transfers and isolating slow I/O traffic, while cache memory reduces the effective memory access time - together improving overall system performance.

Q2. Instruction Execution Cycle and CPU Performance - Concept Map

Develop a comprehensive concept map linking the instruction execution cycle (fetch, decode, execute) with CPU performance parameters such as CPI, clock cycles, and execution time. Show how each stage contributes to overall execution time, and include connections to factors like memory access, instruction complexity, and pipeline stalls. Also, represent how benchmarking is used to evaluate performance.



Explanation of the Concept Map

1. Instruction Execution Cycle

Every instruction passes through **Fetch** (instruction read from memory using the Program Counter), **Decode** (Control Unit interprets the opcode and operands), and **Execute** (ALU operation, memory access, or branch is carried out). The cycle then repeats for the next instruction.

2. Performance Parameters

CPI (Cycles Per Instruction) is the average number of clock cycles each instruction takes. **Clock cycle time** is the inverse of clock rate. **Instruction Count (IC)** is the number of instructions executed. These combine as **Execution Time = IC x CPI x Clock Cycle Time**, the fundamental performance equation.

3. Stage Contribution to Execution Time

The **fetch** stage's duration depends on memory access time (cache hits are fast, misses are slow). The **decode** stage's cycles depend on instruction complexity (simple instructions decode quickly, multi-cycle/complex instructions take longer). The **execute** stage can be delayed by pipeline stalls, directly raising the effective CPI.

4. Contributing Factors

Memory access time (cache hit/miss, RAM latency) affects the fetch stage. **Instruction complexity** affects how many cycles decode/execute need. **Pipeline stalls** - caused by data hazards, control hazards, or structural hazards - introduce bubble cycles that increase the effective CPI and hence the total execution time.

5. Benchmarking

Execution time and CPI are measured and compared using standard benchmarks such as SPEC, MIPS, and FLOPS. Benchmarking runs representative workloads on different systems so that CPI, clock rate, and overall execution time can be objectively compared, giving a fair evaluation of real-world CPU performance.